

Complete SQL and Database Systems Tutorial

From Basic to Advanced — Using MySQL

Prepared for Arif

September 2026

Contents

Introduction: How To Use This Book	5
Chapter 1: What Is a Database?	6
1.1 The Simple Idea	6
1.2 What Is a DBMS?	6
1.3 What Is an RDBMS?	6
1.4 What Is SQL?	6
1.5 Tables, Rows, and Columns	7
1.6 Why Learn MySQL?	7
Chapter 2: Installing MySQL and MySQL Workbench on Arch Linux	8
2.1 A Quick Note on MySQL vs MariaDB on Arch	8
2.2 Option A: Install MariaDB (Recommended, Official Repos)	8
2.3 Option B: Install Official Oracle MySQL from the AUR	9
2.4 Installing MySQL Workbench on Arch Linux	9
2.5 A Few Useful Service Commands	10
Chapter 3: Our Practice Database	11
3.1 Creating the Database	11
3.2 Creating Our Tables	11
3.3 Filling the Tables With Sample Data	12
Chapter 4: SQL Basics — Reading Data (SELECT)	13
4.1 The Simplest Query	13
4.2 Choosing Only Certain Columns	13
4.3 Filtering Rows With WHERE	13
4.4 Sorting Results With ORDER BY	14
4.5 Limiting Results With LIMIT	14
4.6 Removing Duplicates With DISTINCT	14
4.7 Renaming Columns With AS (Alias)	15
4.8 Doing Small Math in a Query	15
Chapter 5: Changing Data (INSERT, UPDATE, DELETE)	16
5.1 INSERT — Adding New Rows	16

5.2 UPDATE — Changing Existing Rows	16
5.3 DELETE — Removing Rows	16
5.4 A Safety Habit: Always SELECT Before You UPDATE/DELETE	17
Chapter 6: Data Types and Constraints	18
6.1 Common MySQL Data Types	18
6.2 What Is a Constraint?	18
6.3 PRIMARY KEY	18
6.4 AUTO_INCREMENT	19
6.5 NOT NULL	19
6.6 UNIQUE	19
6.7 DEFAULT	19
6.8 CHECK	19
6.9 FOREIGN KEY	19
6.10 Changing a Table After It's Created (ALTER TABLE)	20
6.11 Deleting Tables and Databases	20
Chapter 7: Joining Tables (JOIN)	21
7.1 Why Do We Need Joins?	21
7.2 INNER JOIN	21
7.3 LEFT JOIN (LEFT OUTER JOIN)	21
7.4 RIGHT JOIN (RIGHT OUTER JOIN)	21
7.5 FULL OUTER JOIN (MySQL Trick)	22
7.6 CROSS JOIN	22
7.7 SELF JOIN	22
7.8 Joining Three or More Tables	22
Chapter 8: Grouping and Aggregate Functions	24
8.1 What Is an Aggregate Function?	24
8.2 GROUP BY — Aggregates Per Group	24
8.3 HAVING — Filtering Groups	24
8.4 Putting It All Together: Query Order	25
Chapter 9: Subqueries	26
9.1 What Is a Subquery?	26
9.2 Subquery in WHERE	26
9.3 Subquery With IN	26
9.4 Subquery With EXISTS	26
9.5 Subquery in FROM (Derived Table)	26
9.6 Correlated Subqueries	27
Chapter 10: Set Operations, Views, and Indexes	28
10.1 UNION and UNION ALL	28
10.2 INTERSECT and EXCEPT (and MySQL Notes)	28
10.3 Views	28
10.4 Indexes	29
Chapter 11: Database Design and Normalization	30
11.1 Why Design Matters	30

11.2 Entity-Relationship (ER) Modeling	30
11.3 What Is Normalization?	30
Un-normalized Example (The Problem)	30
First Normal Form (1NF)	31
Second Normal Form (2NF)	31
Third Normal Form (3NF)	31
Beyond 3NF (Just Know the Names)	32
11.4 Denormalization (The Opposite, on Purpose)	32
Chapter 12: Advanced SQL — Window (Analytical) Functions	33
12.1 The Big Idea: What Problem Do Window Functions Solve?	33
12.2 The OVER() Clause — The Heart of Window Functions	33
12.3 PARTITION BY — Splitting the Window Into Groups	33
12.4 ORDER BY Inside OVER() — Adding Order to the Window	33
12.5 Ranking Functions: ROW_NUMBER, RANK, DENSE_RANK	34
Practical Example: Find the Top-Paid Employee in Each Department . .	35
12.6 LAG() and LEAD() — Looking at Neighboring Rows	35
Practical Example: Month-Over-Month Comparison	35
12.7 FIRST_VALUE() and LAST_VALUE()	36
12.8 Understanding Window Frames (ROWS/RANGE BETWEEN)	36
12.9 Practical Example: Moving Average (3-Row Moving Average)	37
12.10 Practical Example: Running Total (Cumulative Sum)	37
12.11 Quick Summary Table	37
Chapter 13: Stored Procedures, Functions, and Cursors	38
13.1 Before Cursors: What Is a Stored Procedure?	38
Changing the Statement Delimiter	38
A Procedure With Parameters	38
13.2 What Is a Cursor? (The Core Idea)	39
13.3 Implicit Cursors (Oracle PL/SQL Concept)	39
The MySQL Equivalent Idea	40
13.4 Explicit Cursors — Managing Multi-Row Processing Yourself	40
The Explicit Cursor Lifecycle (Oracle PL/SQL)	40
Cursor FOR Loops (Oracle PL/SQL — the Shortcut)	41
The MySQL Equivalent: Cursors in Stored Procedures	42
13.5 Parameterized Cursors — Passing Arguments Dynamically	43
13.6 Exception Handling With Cursors	44
Oracle PL/SQL: NO_DATA_FOUND and TOO_MANY_ROWS	44
MySQL Equivalent: DECLARE ... HANDLER	44
13.7 Quick Summary: Cursor Concepts, Oracle vs MySQL	45
Chapter 14: Database Triggers	47
14.1 What Is a Trigger?	47
14.2 Trigger Structure — The Basic Pieces	47
14.3 BEFORE vs AFTER vs INSTEAD OF	47
14.4 Row-Level vs Statement-Level Triggers	47
14.5 Correlation Identifiers: NEW and OLD	48
14.6 Writing DML Triggers in MySQL	48

Example 1: BEFORE INSERT Trigger — Validating/Cleaning Data . . .	48
Example 2: BEFORE UPDATE Trigger — Preventing an Invalid Change	48
Example 3: AFTER INSERT Trigger — Data Auditing / History Tracking	49
Example 4: AFTER DELETE Trigger — Keeping a Record of Deleted Data	50
14.7 Practical Applications Recap	50
14.8 Managing Triggers	51
Chapter 15: Stored Functions vs Stored Procedures	52
15.1 The Difference	52
Chapter 16: Transactions and Concurrency (ACID)	53
16.1 What Is a Transaction?	53
16.2 COMMIT and ROLLBACK	53
16.3 ACID — The 4 Properties of a Good Transaction	53
16.4 Isolation Levels (Just the Basics)	54
Chapter 17: Basic Database Security	55
17.1 Users and Privileges	55
17.2 Removing Privileges	55
17.3 Deleting a User	55
17.4 Good Security Habits	55
Chapter 18: Performance Basics	56
18.1 Using EXPLAIN	56
18.2 General Tips	56
Chapter 19: Practice Project — Build a Mini Store Database	57
19.1 Setting Up the Project	57
19.2 Practice Tasks (Try Them Yourself!)	58
19.3 Sample Answers (Check Your Work)	58
Appendix A: Quick Reference Cheat Sheet	60
Basic Queries	60
Table Management	60
Joins	60
Grouping	60
Window Functions	60
Stored Procedure Skeleton	60
Cursor Skeleton (MySQL)	61
Trigger Skeleton (MySQL)	61
Transactions	61
Users and Privileges	61
Appendix B: Where to Go Next	62

Introduction: How To Use This Book

This book teaches you Database Systems and SQL, starting from zero, and going all the way to advanced topics like Cursors, Triggers, and Window Functions.

Rules followed in this book:

- We use **simple English** everywhere. No hard words. If a hard word must be used (like “Normalization”), we explain it in plain words right away.
- We use **MySQL** as our main database software, because it is free, popular, and good for learning.
- Some advanced topics (like Cursors and Triggers) come originally from Oracle’s language called **PL/SQL**. MySQL has its own version of these ideas. In this book, we teach you **both**: the Oracle PL/SQL way (because you asked for it by name) and the MySQL way (because that is the database we are using). This way, you understand the *idea*, and you can use it in *any* database, not just one.
- Every topic has: a simple explanation, a real example, and practice you can try yourself.
- Go in order. Do not skip. Later chapters use ideas from earlier chapters.

By the end of this book, you will be able to:

1. Install MySQL and MySQL Workbench on Arch Linux.
2. Design a database from scratch.
3. Write simple to complex SQL queries.
4. Use Joins, Subqueries, Views, and Indexes.
5. Write Stored Procedures, Functions, Cursors, and Triggers.
6. Use advanced Window Functions like RANK, LAG, LEAD, and running totals.
7. Understand transactions, security, and performance basics.

Let’s begin.

Chapter 1: What Is a Database?

1.1 The Simple Idea

A **database** is just an organized place to store information (data), so you can find it and use it easily later.

Think about a school. The school keeps:

- A list of students (names, roll numbers, classes)
- A list of teachers
- A list of grades/marks
- A list of fees paid

If all this was written on loose paper, finding one student's marks would take forever. A database stores this information in a clean, organized way — usually as **tables** (like spreadsheets with rows and columns) — so a computer program can find, add, change, or remove information in less than a second.

1.2 What Is a DBMS?

DBMS stands for **Database Management System**. It is software that lets you create, read, update, and delete data in a database. Examples of DBMS software:

- MySQL (what we use in this book)
- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- SQLite
- MariaDB

1.3 What Is an RDBMS?

RDBMS stands for **Relational Database Management System**. “Relational” means the data is stored in **tables** (rows and columns), and tables can be **related** (linked) to each other using common values.

Example: A students table and a grades table can be linked using a `student_id` column that appears in both tables.

MySQL is an RDBMS.

1.4 What Is SQL?

SQL stands for **Structured Query Language**. It is the language we use to talk to a database. With SQL, you can:

- Create tables
- Put data into tables
- Read (search/filter) data
- Change data

- Delete data
- Control who can see or change data
- Do advanced calculations on data

SQL is not exactly a “programming language” like Python or Java. It is more like a “question and command language” made just for databases. It reads almost like English:

```
SELECT name FROM students WHERE age > 18;
```

This sentence means: “Show me the name of every row in the students table where age is more than 18.”

1.5 Tables, Rows, and Columns

A **table** looks like this:

student_id	name	age	department
1	Arif	20	CSE
2	Priya	21	EEE
3	Karim	19	CSE

- Each **column** is one piece of information about the thing (like name, age).
- Each **row** is one full record (one full student).
- This whole thing is one **table**, and we might name it students.

1.6 Why Learn MySQL?

MySQL is:

- Free and open-source
- Used by huge companies (Facebook, YouTube, Airbnb use it or used it)
- Easy to learn, but powerful enough for advanced use
- Well-supported: tons of tutorials, tools, and jobs need it
- The base of many web apps (it’s the “M” in the old “LAMP stack”: Linux, Apache, MySQL, PHP)

We will install it soon, and use it for every example in this book.

Chapter 2: Installing MySQL and MySQL Workbench on Arch Linux

This chapter is a step-by-step guide. Type the commands exactly as shown, in your terminal.

2.1 A Quick Note on MySQL vs MariaDB on Arch

Arch Linux’s official repository does **not** ship the official Oracle MySQL server package directly in the normal way some other distros do. Arch offers **MariaDB** in the official repos (MariaDB is a fully compatible fork/replacement of MySQL — almost all MySQL SQL commands work the same way on MariaDB). If you want the exact, official “MySQL” package from Oracle, you install it from the **AUR** (Arch User Repository).

We will show **both options**. Pick Option A if you just want to learn SQL (recommended, simpler). Pick Option B if you specifically need the official Oracle MySQL server.

2.2 Option A: Install MariaDB (Recommended, Official Repos)

Step 1 — Update your system first:

```
sudo pacman -Syu
```

Step 2 — Install the MariaDB package:

```
sudo pacman -S mariadb
```

Step 3 — Initialize the MariaDB data directory (this sets up the initial system files/database):

```
sudo mariadb-install-db --user=mysql --basedir=/usr  
↪ --datadir=/var/lib/mysql
```

Step 4 — Enable and start the MariaDB service using systemd:

```
sudo systemctl enable --now mariadb
```

- enable means “start this automatically every time the computer boots up.”
- --now means “also start it right now.”

Step 5 — Check that it is running:

```
sudo systemctl status mariadb
```

You should see a green “active (running)” message. Press q to exit this screen.

Step 6 — Run the security setup script (sets root password, removes test databases, etc.):

```
sudo mysql_secure_installation
```

It will ask you several yes/no questions. For a learning setup, answer:

- Set root password? **Yes** (choose a password you will remember)
- Remove anonymous users? **Yes**
- Disallow root login remotely? **Yes**
- Remove test database? **Yes**
- Reload privilege tables now? **Yes**

Step 7 — Log in to MySQL/MariaDB to test it:

```
sudo mysql -u root -p
```

Type your root password when asked. If you see a prompt like this, it worked:

```
MariaDB [(none)]>
```

Type EXIT; to leave.

2.3 Option B: Install Official Oracle MySQL from the AUR

If you need the exact Oracle MySQL server (not MariaDB), use an AUR helper like yay or paru. If you don't have one yet, install yay first:

```
sudo pacman -S --needed base-devel git
git clone https://aur.archlinux.org/yay.git
cd yay
makepkg -si
```

Now install MySQL from the AUR:

```
yay -S mysql
```

Then initialize and start it, similar to MariaDB:

```
sudo mysql_install_db --user=mysql --basedir=/usr --datadir=/var/lib/mysql
sudo systemctl enable --now mysqld
sudo mysql_secure_installation
```

Note: AUR packages are community-maintained, not official Arch packages, so they can sometimes lag behind or need small manual fixes. For most learners, **Option A (MariaDB)** is easier and just as good for learning SQL — everything in this book will work the same way.

2.4 Installing MySQL Workbench on Arch Linux

MySQL Workbench is a graphical (visual) tool. Instead of typing every command in a black terminal window, you get a nice window where you can see tables, click buttons, and write SQL with color highlighting.

Step 1 — Install it from the official Arch repos:

```
sudo pacman -S mysql-workbench
```

If it is not found in your repo (sometimes it moves to AUR depending on your Arch version/mirror), use an AUR helper instead:

```
yay -S mysql-workbench
```

Step 2 — Open it. You can launch it from your application menu, or from the terminal by typing:

```
mysql-workbench
```

Step 3 — Create a new connection in Workbench:

1. Open MySQL Workbench.
2. Click the **+** icon next to “MySQL Connections.”
3. Fill in:
 - Connection Name: Local Learning DB (or any name)
 - Hostname: 127.0.0.1
 - Port: 3306
 - Username: root
4. Click **Test Connection**, type your root password, and click **OK**.
5. Click **OK** to save the connection.
6. Double-click the new connection to open a SQL editor window.

Now you have two ways to practice everything in this book:

- **Terminal way:** `sudo mysql -u root -p` then type SQL commands.
- **Workbench way:** Open Workbench, connect, and type SQL commands in the query tab, then press the lightning bolt icon to run them.

Both do the exact same thing. Use whichever you feel comfortable with. This book’s examples work in both.

2.5 A Few Useful Service Commands

```
# Start the database service
sudo systemctl start mariadb

# Stop the database service
sudo systemctl stop mariadb

# Restart the database service
sudo systemctl restart mariadb

# Check if it is running
sudo systemctl status mariadb
```

(Replace mariadb with mysqld if you installed Option B, the AUR MySQL package.)

Chapter 3: Our Practice Database

Through most of this book, we will use one running example: a small **Company Database**. This helps you see how ideas connect, instead of jumping between random examples.

3.1 Creating the Database

```
-- Create a new database (a container that holds all our tables)
CREATE DATABASE company_db;

-- Tell MySQL: "use this database for all commands from now on"
USE company_db;
```

3.2 Creating Our Tables

```
CREATE TABLE departments (
    dept_id      INT PRIMARY KEY AUTO_INCREMENT,
    dept_name    VARCHAR(50) NOT NULL,
    location     VARCHAR(50)
);

CREATE TABLE employees (
    emp_id       INT PRIMARY KEY AUTO_INCREMENT,
    first_name   VARCHAR(50) NOT NULL,
    last_name    VARCHAR(50) NOT NULL,
    email        VARCHAR(100) UNIQUE,
    hire_date    DATE,
    salary       DECIMAL(10,2),
    dept_id      INT,
    manager_id   INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id),
    FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

CREATE TABLE projects (
    project_id   INT PRIMARY KEY AUTO_INCREMENT,
    project_name VARCHAR(100) NOT NULL,
    budget       DECIMAL(12,2),
    dept_id      INT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);

CREATE TABLE works_on (
    emp_id       INT,
    project_id    INT,
```

```

hours_spent DECIMAL(6,2),
PRIMARY KEY (emp_id, project_id),
FOREIGN KEY (emp_id) REFERENCES employees(emp_id),
FOREIGN KEY (project_id) REFERENCES projects(project_id)
);

```

Don't worry if some words here (PRIMARY KEY, FOREIGN KEY, AUTO_INCREMENT) look new. We explain every one of them in this chapter and the next. For now, just know: we made 4 tables that are connected to each other.

3.3 Filling the Tables With Sample Data

```

INSERT INTO departments (dept_name, location) VALUES
('Engineering', 'Dhaka'),
('Sales', 'Chittagong'),
('HR', 'Dhaka');

```

```

INSERT INTO employees (first_name, last_name, email, hire_date, salary,
↳ dept_id, manager_id) VALUES
('Arif', 'Hossain', 'arif@company.com', '2021-03-15', 55000.00, 1, NULL),
('Priya', 'Das', 'priya@company.com', '2020-06-01', 75000.00, 1, NULL),
('Karim', 'Uddin', 'karim@company.com', '2022-01-10', 45000.00, 1, 2),
('Nusrat', 'Jahan', 'nusrat@company.com', '2019-11-20', 60000.00, 2, NULL),
('Sabbir', 'Ahmed', 'sabbir@company.com', '2023-02-05', 40000.00, 2, 4),
('Farhana', 'Akter', 'farhana@company.com', '2021-08-17', 50000.00, 3,
↳ NULL);

```

```

INSERT INTO projects (project_name, budget, dept_id) VALUES
('Website Revamp', 200000.00, 1),
('Mobile App', 350000.00, 1),
('Sales Portal', 150000.00, 2);

```

```

INSERT INTO works_on (emp_id, project_id, hours_spent) VALUES
(1, 1, 120.5),
(1, 2, 80.0),
(2, 1, 150.0),
(3, 2, 200.0),
(4, 3, 100.0),
(5, 3, 90.5);

```

Keep this schema in mind. We will use it again and again in later chapters (Joins, Group By, Window Functions, Triggers, etc). If you want, open MySQL Workbench or your terminal right now and type all of the commands above, so your database is ready for the rest of the book.

Chapter 4: SQL Basics — Reading Data (SELECT)

4.1 The Simplest Query

```
SELECT * FROM employees;
```

- SELECT means “give me.”
- * means “all columns.”
- FROM employees means “from the employees table.”

So this line means: “Give me all columns, from the employees table.” It shows every row, every column.

4.2 Choosing Only Certain Columns

```
SELECT first_name, last_name, salary FROM employees;
```

This shows only 3 columns, not all of them. Always pick only the columns you need — it’s faster and cleaner.

4.3 Filtering Rows With WHERE

```
SELECT first_name, salary  
FROM employees  
WHERE salary > 50000;
```

This only shows employees whose salary is more than 50000.

Common comparison operators:

Operator	Meaning
=	equal to
!= or <>	not equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

Combining conditions:

```
-- AND: both conditions must be true  
SELECT * FROM employees WHERE salary > 40000 AND dept_id = 1;  
  
-- OR: at least one condition must be true  
SELECT * FROM employees WHERE dept_id = 2 OR dept_id = 3;
```

```
-- NOT: flips true/false
SELECT * FROM employees WHERE NOT dept_id = 1;
```

Other useful filter tools:

```
-- BETWEEN: check a range (inclusive of both ends)
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 60000;

-- IN: check if value matches one of a list
SELECT * FROM employees WHERE dept_id IN (1, 3);

-- LIKE: pattern matching for text
-- % means "any number of any characters"
-- _ means "exactly one character"
SELECT * FROM employees WHERE first_name LIKE 'A%'; -- starts with A
SELECT * FROM employees WHERE email LIKE '%company.com'; -- ends with
    ↪ company.com

-- IS NULL / IS NOT NULL: checking for missing values
SELECT * FROM employees WHERE manager_id IS NULL;
```

4.4 Sorting Results With ORDER BY

```
-- Ascending order (smallest to largest) - this is the default
SELECT first_name, salary FROM employees ORDER BY salary ASC;

-- Descending order (largest to smallest)
SELECT first_name, salary FROM employees ORDER BY salary DESC;

-- Sort by more than one column
SELECT * FROM employees ORDER BY dept_id ASC, salary DESC;
```

4.5 Limiting Results With LIMIT

```
-- Show only the first 3 rows
SELECT * FROM employees LIMIT 3;

-- Skip the first 2 rows, then show the next 3 (useful for "pages" of
    ↪ results)
SELECT * FROM employees LIMIT 3 OFFSET 2;
```

4.6 Removing Duplicates With DISTINCT

```
SELECT DISTINCT dept_id FROM employees;
```

This shows each different dept_id value only once, even if many employees share it.

4.7 Renaming Columns With AS (Alias)

```
SELECT first_name AS name, salary AS monthly_pay  
FROM employees;
```

This does not change the real table. It just changes what the *result* looks like, to make it easier to read.

4.8 Doing Small Math in a Query

```
SELECT first_name, salary, salary * 12 AS yearly_salary  
FROM employees;
```

You can do +, -, *, / right inside SELECT.

Chapter 5: Changing Data (INSERT, UPDATE, DELETE)

5.1 INSERT — Adding New Rows

```
-- Add one row
INSERT INTO departments (dept_name, location)
VALUES ('Marketing', 'Sylhet');

-- Add many rows at once
INSERT INTO departments (dept_name, location) VALUES
('Finance', 'Dhaka'),
('Support', 'Khulna');
```

If a column has a default rule or auto-increments (like dept_id), you can skip it in the column list, and MySQL will fill it in for you.

5.2 UPDATE — Changing Existing Rows

```
UPDATE employees
SET salary = 58000.00
WHERE emp_id = 1;
```

Very important warning: If you forget the WHERE part, MySQL will update **every single row** in the table. Always double check your WHERE clause before running an UPDATE.

```
-- Update more than one column at once
UPDATE employees
SET salary = salary * 1.10, dept_id = 2
WHERE emp_id = 3;
```

This raises employee 3's salary by 10% and moves them to department 2.

5.3 DELETE — Removing Rows

```
DELETE FROM employees
WHERE emp_id = 6;
```

Same warning applies here. DELETE FROM employees; with no WHERE will delete every row in the table (but keep the empty table itself).

To empty a table fast and reset auto-increment numbers, use:

```
TRUNCATE TABLE employees;
```

TRUNCATE is faster than DELETE for wiping a whole table, but it cannot be undone with ROLLBACK in most setups, and it resets the AUTO_INCREMENT counter. Use it carefully.

5.4 A Safety Habit: Always SELECT Before You UPDATE/DELETE

Good habit: before running an UPDATE or DELETE, run the same WHERE condition with a SELECT first, to see exactly which rows will be affected.

```
-- Step 1: check first
SELECT * FROM employees WHERE dept_id = 2 AND salary < 45000;

-- Step 2: if that looks correct, THEN update
UPDATE employees SET salary = 45000 WHERE dept_id = 2 AND salary < 45000;
```

Chapter 6: Data Types and Constraints

6.1 Common MySQL Data Types

Category	Type	Meaning
Numbers	INT	Whole number (e.g., 5, -20, 1000)
Numbers	DECIMAL(p, s)	Exact decimal number. p = total digits, s = digits after the dot. Good for money.
Numbers	FLOAT / DOUBLE	Approximate decimal number. Do not use for money.
Text	VARCHAR(n)	Text with a maximum length of n characters. Most common text type.
Text	CHAR(n)	Fixed-length text, always exactly n characters (padded with spaces).
Text	TEXT	Very long text (like an article or comment).
Date/Time	DATE	A date only, like 2026-09-17.
Date/Time	TIME	A time only, like 14:30:00.
Date/Time	DATETIME	Date and time together.
Date/Time	TIMESTAMP	Like DATETIME, but often auto-updates and is tied to time zones.
Boolean	BOOLEAN (really TINYINT(1))	True (1) or False (0).

6.2 What Is a Constraint?

A **constraint** is a rule that MySQL enforces automatically on a column or table, so bad data cannot get in.

6.3 PRIMARY KEY

A **primary key** is a column (or group of columns) that makes each row unique. No two rows can have the same primary key value, and it cannot be NULL (empty).

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(50)  
);
```

Almost every table should have a primary key. It is how we point at “exactly this one row.”

6.4 AUTO_INCREMENT

This tells MySQL: “fill this number in automatically, and make each new row’s number one bigger than the last.”

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY AUTO_INCREMENT,  
    name        VARCHAR(50)  
);  
  
INSERT INTO customers (name) VALUES ('Rahim'); -- gets customer_id = 1  
↪ automatically  
INSERT INTO customers (name) VALUES ('Karim'); -- gets customer_id = 2  
↪ automatically
```

6.5 NOT NULL

Forces a column to always have a value — it can never be left empty.

```
name VARCHAR(50) NOT NULL
```

6.6 UNIQUE

Forces every value in a column to be different from every other value (but, unlike primary key, it CAN be NULL, and a table can have many UNIQUE columns).

```
email VARCHAR(100) UNIQUE
```

6.7 DEFAULT

If no value is given, MySQL fills in this default value automatically.

```
status VARCHAR(20) DEFAULT 'active'
```

6.8 CHECK

Forces a value to pass a test/condition before it is allowed.

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    price DECIMAL(10,2) CHECK (price > 0)  
);
```

Now MySQL will reject any attempt to insert a product with a price of 0 or less.

6.9 FOREIGN KEY

A **foreign key** links a column in one table to the primary key of another table. It makes sure you cannot reference something that does not exist.

```
CREATE TABLE orders (
  order_id    INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT,
  order_date  DATE,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Now, you cannot insert an order with a `customer_id` that does not exist in the `customers` table. This keeps your data honest and connected correctly. This idea — using foreign keys to connect tables — is the heart of “relational” databases, and it’s what makes JOIN (which we learn next) possible.

6.10 Changing a Table After It’s Created (ALTER TABLE)

```
-- Add a new column
ALTER TABLE employees ADD COLUMN phone VARCHAR(20);

-- Change a column's type
ALTER TABLE employees MODIFY COLUMN phone VARCHAR(30);

-- Rename a column
ALTER TABLE employees RENAME COLUMN phone TO phone_number;

-- Remove a column
ALTER TABLE employees DROP COLUMN phone_number;

-- Add a constraint after the table exists
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);
```

6.11 Deleting Tables and Databases

```
DROP TABLE employees;      -- deletes the whole table, including its data
                             ⇨ and structure
DROP DATABASE company_db;   -- deletes the whole database, including every
                             ⇨ table inside it
```

Be very careful with DROP. There is no “undo” button in SQL by default.

Chapter 7: Joining Tables (JOIN)

7.1 Why Do We Need Joins?

Our data is spread across many tables (employees, departments, projects, works_on). Most of the time, we need to combine information from more than one table into a single result. That is what a **JOIN** does.

7.2 INNER JOIN

INNER JOIN gives you only the rows that **match** in both tables.

```
SELECT e.first_name, e.last_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

- e and d are **aliases** (short nicknames) for employees and departments. This saves typing and makes the query easier to read.
- ON e.dept_id = d.dept_id tells MySQL exactly how the two tables are connected.
- If an employee has a dept_id that does not exist in departments (or is NULL), that employee will **not** show up in the result, because there is no “match.”

7.3 LEFT JOIN (LEFT OUTER JOIN)

LEFT JOIN gives you **every row from the left table**, plus matching data from the right table. If there is no match, the right table’s columns show as NULL.

```
SELECT d.dept_name, e.first_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

This shows every department, even a department with **zero** employees (it would show NULL for first_name).

7.4 RIGHT JOIN (RIGHT OUTER JOIN)

RIGHT JOIN is the mirror image of LEFT JOIN — it keeps every row from the **right** table.

```
SELECT d.dept_name, e.first_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
```

This gives the exact same result as the LEFT JOIN example above, just written the other way around. In real life, most people just use LEFT JOIN and swap the table order, instead of using RIGHT JOIN, because it’s easier to read.

7.5 FULL OUTER JOIN (MySQL Trick)

Many databases (like PostgreSQL) support FULL OUTER JOIN directly (it keeps every row from both tables, matched or not). **MySQL does not support FULL OUTER JOIN directly.** We can build the same result using LEFT JOIN + UNION + RIGHT JOIN:

```
SELECT d.dept_name, e.first_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id

UNION

SELECT d.dept_name, e.first_name
FROM departments d
RIGHT JOIN employees e ON d.dept_id = e.dept_id;
```

UNION combines both result sets and automatically removes exact duplicate rows.

7.6 CROSS JOIN

CROSS JOIN combines **every row** of one table with **every row** of another table (all possible pairs). It is rarely used on purpose, but good to know.

```
SELECT e.first_name, p.project_name
FROM employees e
CROSS JOIN projects p;
```

If employees has 6 rows and projects has 3 rows, this returns 18 rows (6×3).

7.7 SELF JOIN

A **self join** joins a table to itself. It's useful when a table refers to itself, like our employees table, where manager_id points to another row in the same table.

```
SELECT e.first_name AS employee_name, m.first_name AS manager_name
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

Here, e and m are both "the employees table," but treated as two separate copies during the join, so we can compare an employee to their manager (who is also just a row in the same table).

7.8 Joining Three or More Tables

You can chain as many joins as you need:

```
SELECT e.first_name, d.dept_name, p.project_name, w.hours_spent
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
```

```
JOIN works_on w ON e.emp_id = w.emp_id  
JOIN projects p ON w.project_id = p.project_id;
```

(Note: plain JOIN means the same thing as INNER JOIN.)

This single query pulls together data from **four** different tables at once, showing which employee, in which department, worked on which project, for how many hours.

Chapter 8: Grouping and Aggregate Functions

8.1 What Is an Aggregate Function?

An **aggregate function** takes many rows and turns them into **one single answer**, like a total, an average, or a count.

Function	Meaning
COUNT()	How many rows
SUM()	Add up all values
AVG()	Average (mean) value
MIN()	Smallest value
MAX()	Largest value

```
SELECT COUNT(*) FROM employees;           -- how many employees total
SELECT AVG(salary) FROM employees;        -- average salary of everyone
SELECT MAX(salary), MIN(salary) FROM employees; -- highest and lowest
    ↳ salary
SELECT SUM(budget) FROM projects;         -- total budget across all
    ↳ projects
```

8.2 GROUP BY — Aggregates Per Group

GROUP BY splits your rows into groups, and then runs the aggregate function **separately for each group**.

```
SELECT dept_id, COUNT(*) AS num_employees, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;
```

This gives you one row **per department**, showing how many employees are in it, and their average salary. Without GROUP BY, you'd only get one row for the *entire* table.

Rule to remember: every column in your SELECT list must either be (a) inside an aggregate function like SUM(), or (b) listed in the GROUP BY clause. Otherwise MySQL doesn't know which value to show for that column, since each group can contain many rows.

8.3 HAVING — Filtering Groups

WHERE filters rows **before** grouping happens. HAVING filters groups **after** grouping happens. You cannot use aggregate functions inside WHERE, but you can inside HAVING.

```
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > 50000;
```

This only shows departments whose *average* salary is above 50000 — a calculation that only makes sense *after* grouping.

You can combine both:

```
SELECT dept_id, COUNT(*) AS num_employees
FROM employees
WHERE hire_date >= '2020-01-01'    -- filter rows first (only recent hires)
GROUP BY dept_id
HAVING COUNT(*) >= 2;              -- then filter groups (only depts with 2+
    ↳ such hires)
```

8.4 Putting It All Together: Query Order

MySQL actually runs a query in this logical order, even though we *write* it in a different order:

1. FROM — pick the table(s)
2. JOIN — combine with other tables
3. WHERE — filter individual rows
4. GROUP BY — group the remaining rows
5. HAVING — filter the groups
6. SELECT — choose which columns/calculations to show
7. ORDER BY — sort the final result
8. LIMIT — cut down to a certain number of rows

Understanding this order helps explain *why* WHERE can't use aggregate functions (they don't exist yet at that step), but HAVING can.

Chapter 9: Subqueries

9.1 What Is a Subquery?

A **subquery** is a query written *inside* another query. It runs first, and its result is used by the outer (main) query.

9.2 Subquery in WHERE

```
-- Find employees who earn more than the average salary
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Here, (SELECT AVG(salary) FROM employees) runs first, produces one number (the average), and then the outer query compares each employee's salary against that number.

9.3 Subquery With IN

```
-- Find employees who work on any project with budget > 200000
SELECT first_name
FROM employees
WHERE emp_id IN (
    SELECT w.emp_id
    FROM works_on w
    JOIN projects p ON w.project_id = p.project_id
    WHERE p.budget > 200000
);
```

9.4 Subquery With EXISTS

EXISTS checks if a subquery returns **any** rows at all (true/false), without caring about the actual values. It's often faster than IN for large tables.

```
SELECT d.dept_name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id
);
```

This shows departments that have at least one employee.

9.5 Subquery in FROM (Derived Table)

You can even use a subquery as if it were a table:

```

SELECT dept_id, avg_salary
FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) AS dept_averages
WHERE avg_salary > 45000;

```

Here, the inner query builds a small temporary table of department averages, and the outer query filters that temporary table. Every derived table (subquery in FROM) must have an alias — that’s why we wrote AS dept_averages.

9.6 Correlated Subqueries

A **correlated subquery** is a subquery that uses a value from the *outer* query — meaning it runs once *per row* of the outer query, not just once overall.

```

-- Find employees who earn more than the average salary IN THEIR OWN
↪ department
SELECT e1.first_name, e1.salary, e1.dept_id
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id  -- this line links inner query to outer
    ↪ row
);

```

This is more powerful (and slower) than a normal subquery, because the inner query re-runs for every single row of the outer query, using that row’s dept_id each time.

Chapter 10: Set Operations, Views, and Indexes

10.1 UNION and UNION ALL

UNION combines the results of two SELECT statements into one list, and removes exact duplicate rows. UNION ALL does the same but **keeps** duplicates (and is faster, since it skips the duplicate-checking step).

Both queries being combined must have the **same number of columns**, in a compatible order/type.

```
SELECT first_name FROM employees WHERE dept_id = 1
UNION
SELECT first_name FROM employees WHERE dept_id = 2;

-- keeps duplicate names if they exist in both groups
SELECT first_name FROM employees WHERE dept_id = 1
UNION ALL
SELECT first_name FROM employees WHERE dept_id = 2;
```

10.2 INTERSECT and EXCEPT (and MySQL Notes)

Newer versions of MySQL (8.0.31 and later) support INTERSECT (common rows in both queries) and EXCEPT (rows in the first query but not the second) directly. If your MySQL version is older, you can fake them:

```
-- INTERSECT alternative (rows that appear in BOTH queries)
SELECT emp_id FROM works_on WHERE project_id = 1
AND emp_id IN (SELECT emp_id FROM works_on WHERE project_id = 2);

-- EXCEPT alternative (rows in query A but NOT in query B)
SELECT emp_id FROM employees
WHERE emp_id NOT IN (SELECT emp_id FROM works_on);
```

10.3 Views

A **view** is a saved query that behaves like a virtual table. It does not store data itself — every time you use it, it re-runs its underlying query.

```
CREATE VIEW high_earners AS
SELECT first_name, last_name, salary, dept_id
FROM employees
WHERE salary > 50000;

-- Now use it just like a table:
SELECT * FROM high_earners WHERE dept_id = 1;
```

Why use views?

- **Simplicity:** hide a complicated join/query behind a simple name.

- **Security:** give someone access to a view (e.g., only certain columns), without giving them access to the full underlying table.
- **Consistency:** everyone uses the same “official” version of a calculation.

To update or remove a view:

```
CREATE OR REPLACE VIEW high_earners AS
SELECT first_name, last_name, salary FROM employees WHERE salary > 55000;

DROP VIEW high_earners;
```

10.4 Indexes

An **index** is a special lookup structure that makes searching a table much faster — similar to the index at the back of a textbook, which helps you find a topic without reading every page.

```
CREATE INDEX idx_employee_lastname ON employees(last_name);
```

Now, a query like this will run much faster on a large table:

```
SELECT * FROM employees WHERE last_name = 'Hossain';
```

Important trade-off: indexes make **reading (SELECT)** faster, but make **writing (INSERT/UPDATE/DELETE)** slightly slower, because MySQL has to update the index every time data changes. Don’t add indexes on every column “just in case” — add them on columns you frequently search or join on.

A PRIMARY KEY and a UNIQUE constraint both automatically create an index behind the scenes.

To remove an index:

```
DROP INDEX idx_employee_lastname ON employees;
```

To see what indexes a table has:

```
SHOW INDEXES FROM employees;
```

Chapter 11: Database Design and Normalization

11.1 Why Design Matters

Before making tables, a good database designer plans things out. Bad design leads to duplicate data, wasted storage, and confusing bugs later. Good design starts with two big steps: **ER modeling** and **Normalization**.

11.2 Entity-Relationship (ER) Modeling

An **entity** is a “thing” you want to store data about (a Student, a Product, an Employee). An **ER diagram** is a simple drawing that shows:

- **Entities** (drawn as boxes) — these usually become tables.
- **Attributes** (properties of an entity, like name, age) — these usually become columns.
- **Relationships** (how entities connect, like “an Employee *works in* a Department”) — these usually become foreign keys.

Types of relationships:

- **One-to-One (1:1)**: one row in Table A matches exactly one row in Table B (e.g., one Person has one Passport).
- **One-to-Many (1:M)**: one row in Table A matches many rows in Table B (e.g., one Department has many Employees). This is the most common relationship, handled with a simple foreign key.
- **Many-to-Many (M:N)**: many rows in Table A match many rows in Table B (e.g., many Employees work on many Projects). This needs a **junction/bridge table** in between — that’s exactly what our `works_on` table is! It sits between employees and projects, holding a pair of foreign keys, to represent the many-to-many link.

11.3 What Is Normalization?

Normalization is a step-by-step process of organizing tables to reduce **data duplication** and avoid weird update problems. We do this by following a series of rules, called “Normal Forms” (1NF, 2NF, 3NF, and so on).

Un-normalized Example (The Problem)

Imagine one giant messy table:

order_id	customer_name	product_names	product_prices
1	Arif	Pen, Notebook	10, 50

Problems: - Multiple values crammed into one cell (Pen, Notebook) — hard to search or calculate with. - If “Arif” changes his name, we might need to update it in a hundred different rows. - Wastes space by repeating “Arif” every time he places an order.

First Normal Form (1NF)

Rule: Every column must hold only **one single value** (no lists/multiple values in one cell). Every row must be unique.

Fix: split the multiple products into separate rows.

order_id	customer_name	product_name	product_price
1	Arif	Pen	10
1	Arif	Notebook	50

Second Normal Form (2NF)

Rule: Must already be in 1NF. Also, every non-key column must depend on the **whole** primary key, not just part of it. (This rule mostly matters when the primary key is made of more than one column.)

In our example, `customer_name` depends only on `order_id`, not on `product_name`. So we split it further:

orders table:

order_id	customer_name
1	Arif

order_items table:

order_id	product_name	product_price
1	Pen	10
1	Notebook	50

Third Normal Form (3NF)

Rule: Must already be in 2NF. Also, non-key columns must not depend on *other non-key columns* (only on the primary key). This is sometimes summarized as: “no transitive dependency.”

Suppose our `order_items` table also had a `product_category` column, and `product_category` always depends on `product_name` (not on `order_id` directly). That’s a transitive dependency — we should move product details into their own products table:

products table:

product_id	product_name	product_price	product_category
1	Pen	10	Stationery

product_id	product_name	product_price	product_category
2	Notebook	50	Stationery

order_items table (now just links orders to products):

order_id	product_id	quantity
1	1	3
1	2	1

Now, if a product's price changes, we update it in **one place** (the products table), and every order automatically reflects the correct linked data.

Beyond 3NF (Just Know the Names)

There are further normal forms (BCNF, 4NF, 5NF) for very specific tricky cases, but for most real projects, reaching **3NF** is considered “good enough” and standard practice.

11.4 Denormalization (The Opposite, on Purpose)

Sometimes, for **speed**, designers intentionally break normalization rules a little — for example, storing a calculated `total_price` directly in the orders table, instead of re-calculating it from `order_items` every time. This is called **denormalization**. It trades a little bit of duplicate data for faster reads. Use it carefully, and only after your basic design is already normalized.

Chapter 12: Advanced SQL — Window (Analytical) Functions

This is one of the most powerful and most asked-about topics in modern SQL. Take your time here.

12.1 The Big Idea: What Problem Do Window Functions Solve?

We already learned `GROUP BY`. The problem with `GROUP BY` is: it **collapses** many rows into one row per group. Sometimes we don't want that — we want to do a calculation **across a group of rows**, but still **see every individual row** in the output.

Example: “Show me every employee, AND their department’s average salary next to them” — without hiding any employee.

`GROUP BY` cannot do this alone (it would hide individual employees). **Window functions** can.

12.2 The `OVER()` Clause — The Heart of Window Functions

Every window function uses the keyword `OVER()`. Think of `OVER()` as saying: “do this calculation over a *window* (a slice) of rows, related to the current row — but still show me every row.”

```
SELECT first_name, dept_id, salary,  
       AVG(salary) OVER () AS company_avg_salary  
FROM employees;
```

Here, `AVG(salary) OVER ()` calculates the average salary of **the entire table**, but shows it next to **every single row**, without collapsing anything. Compare that to plain `GROUP BY`, which would give you only 1 row.

12.3 `PARTITION BY` — Splitting the Window Into Groups

`PARTITION BY` splits the rows into groups (partitions), and the window function is calculated **separately for each group** — but every row is still shown.

```
SELECT first_name, dept_id, salary,  
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg_salary  
FROM employees;
```

Now each employee sees their **own department’s** average salary, right next to their own row. This is the “show every row, plus a group calculation” trick that `GROUP BY` alone cannot do.

12.4 `ORDER BY` Inside `OVER()` — Adding Order to the Window

Adding `ORDER BY` inside `OVER()` changes the window to be **running / cumulative**, meaning it only looks at rows *up to* the current row (in the given order), not the whole partition.

```
SELECT first_name, dept_id, hire_date, salary,
       SUM(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS
       ↳ running_total
FROM employees;
```

This gives a **running total** of salary, department by department, ordered by hire date — each row adds its own salary to the sum of everyone hired before it (in the same department).

12.5 Ranking Functions: ROW_NUMBER, RANK, DENSE_RANK

These three functions all give a “rank number” to each row, but they behave differently, especially when there are **ties** (equal values).

```
SELECT first_name, dept_id, salary,
       ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
       ↳ row_num,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
       ↳ rank_num,
       DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
       ↳ dense_rank_num
FROM employees;
```

Imagine 4 employees in one department with salaries: 90000, 90000, 80000, 70000 (two people are tied at the top).

salary	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
90000	2	1	1
80000	3	3	2
70000	4	4	3

- **ROW_NUMBER()**: always gives a unique, ever-increasing number (1, 2, 3, 4...), even for ties. It just numbers the rows in order — it never repeats a number.
- **RANK()**: gives the *same* rank to tied rows, but then **skips** numbers afterward (notice it jumps from 1 straight to 3).
- **DENSE_RANK()**: gives the *same* rank to tied rows too, but does **not** skip any numbers afterward (goes 1, 1, 2, 3 — no gaps).

When to use which: - Use ROW_NUMBER() when you need a truly unique number per row — for example, picking “exactly the top 1 row per group” (like the single highest-paid employee per department, with no ties allowed in the result). - Use RANK() when ties should share the same rank, and you’re okay with gaps (like a race: two people tie for 1st place, so the next person is 3rd place, not 2nd). - Use DENSE_RANK() when ties should share the same rank, but you want a clean, gap-free sequence of ranks (like “how many different distinct salary levels are above this one”).

Practical Example: Find the Top-Paid Employee in Each Department

```
SELECT first_name, dept_id, salary
FROM (
    SELECT first_name, dept_id, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS
           ↪ rn
    FROM employees
) AS ranked
WHERE rn = 1;
```

We wrap the window function in a subquery, then filter with `WHERE rn = 1` in the outer query. (You cannot put a window function directly inside a `WHERE` clause in the same query — it has to be filtered one level “outside,” in the outer query, like this.)

12.6 LAG() and LEAD() — Looking at Neighboring Rows

`LAG()` looks at a **previous** row’s value. `LEAD()` looks at a **following** row’s value. Both are extremely useful for comparing a row to the row “before” or “after” it, once the rows are ordered.

```
SELECT first_name, hire_date, salary,
       LAG(salary, 1) OVER (ORDER BY hire_date) AS prev_employee_salary,
       LEAD(salary, 1) OVER (ORDER BY hire_date) AS next_employee_salary
FROM employees;
```

- `LAG(salary, 1)` means “give me the salary value from **1 row before** this one” (in `hire_date` order).
- `LEAD(salary, 1)` means “give me the salary value from **1 row after** this one.”
- The 1 is how many rows to look back/forward — you can change it to 2, 3, etc.
- The very first row has no “previous” row, so `LAG()` returns `NULL` there (unless you give it a default, explained next).

You can give `LAG/LEAD` a default value for when there is no previous/next row:

```
LAG(salary, 1, 0) OVER (ORDER BY hire_date) -- returns 0 instead of NULL,
↪ if no previous row exists
```

Practical Example: Month-Over-Month Comparison

A very common real use: comparing this month’s sales to last month’s sales.

```
SELECT sale_month, total_sales,
       LAG(total_sales, 1) OVER (ORDER BY sale_month) AS
       ↪ previous_month_sales,
       total_sales - LAG(total_sales, 1) OVER (ORDER BY sale_month) AS
       ↪ change_from_last_month
FROM monthly_sales;
```

12.7 FIRST_VALUE() and LAST_VALUE()

These grab the very **first** or very **last** value within a window (partition), based on the given order.

```
SELECT first_name, dept_id, salary,
       FIRST_VALUE(first_name) OVER (PARTITION BY dept_id ORDER BY salary
        ↪ DESC) AS highest_paid_in_dept,
       LAST_VALUE(first_name) OVER (
        PARTITION BY dept_id ORDER BY salary DESC
        RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
        ) AS lowest_paid_in_dept
FROM employees;
```

Important trap: by default, LAST_VALUE() does *not* look at the whole partition — it only looks at rows “up to the current row” (because of a default window frame, explained next). This is why, in the example above, we had to manually add RANGE BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING — this tells MySQL “look at the WHOLE partition, from the very first row to the very last row,” not just up to the current row.

12.8 Understanding Window Frames (ROWS/RANGE BETWEEN)

A **window frame** is the exact slice of rows the function looks at, *within* the current partition. By default, when you use ORDER BY inside OVER(), MySQL uses this default frame:

RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

This means: “from the start of the partition, up to (and including) the current row.” That’s exactly why SUM() OVER (PARTITION BY ... ORDER BY ...) gives you a **running total**, instead of the full group total.

You can set a custom frame using ROWS BETWEEN:

```
-- Look only at the current row and the 2 rows before it (3 rows total)
SUM(salary) OVER (ORDER BY hire_date ROWS BETWEEN 2 PRECEDING AND CURRENT
 ↪ ROW)

-- Look at the current row, 1 row before, and 1 row after (3 rows total)
AVG(salary) OVER (ORDER BY hire_date ROWS BETWEEN 1 PRECEDING AND 1
 ↪ FOLLOWING)

-- Look at the WHOLE partition, ignoring row position completely
SUM(salary) OVER (PARTITION BY dept_id ROWS BETWEEN UNBOUNDED PRECEDING AND
 ↪ UNBOUNDED FOLLOWING)
```

Keywords used inside ROWS BETWEEN ... AND ...:

- UNBOUNDED PRECEDING — all the way to the first row of the partition
- N PRECEDING — N rows before the current row

- CURRENT ROW — the current row itself
- N FOLLOWING — N rows after the current row
- UNBOUNDED FOLLOWING — all the way to the last row of the partition

12.9 Practical Example: Moving Average (3-Row Moving Average)

A **moving average** smooths out data by averaging a small “window” of nearby rows, instead of just one row — this is very common in sales/finance/stock analysis.

```
SELECT sale_date, daily_sales,
       AVG(daily_sales) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg_3_day
FROM daily_sales_table;
```

For each day, this averages: today’s sales + the 2 days before it = a smooth 3-day moving average, without collapsing any row.

12.10 Practical Example: Running Total (Cumulative Sum)

```
SELECT sale_date, daily_sales,
       SUM(daily_sales) OVER (ORDER BY sale_date) AS running_total_sales
FROM daily_sales_table;
```

Every row shows the total sales **so far**, up to and including that day. This is one of the most common uses of window functions in dashboards and reports.

12.11 Quick Summary Table

Function	What it does
ROW_NUMBER()	Unique increasing number per row, no ties
RANK()	Rank with ties sharing a number, gaps after ties
DENSE_RANK()	Rank with ties sharing a number, no gaps
LAG(col, n)	Value from n rows before the current row
LEAD(col, n)	Value from n rows after the current row
FIRST_VALUE(col)	First value in the window
LAST_VALUE(col)	Last value in the window (careful with frame default!)
SUM/AVG/COUNT/MIN/MAX() OVER(...)	Aggregate calculated across a window, without collapsing rows

Chapter 13: Stored Procedures, Functions, and Cursors

13.1 Before Cursors: What Is a Stored Procedure?

A **stored procedure** is a saved block of SQL code (with logic, like IF statements and loops) that lives inside the database, and can be called (run) by name, again and again.

Important note on PL/SQL vs MySQL: “PL/SQL” is Oracle Database’s own procedural language. MySQL has its **own** procedural language for stored procedures/functions, which looks a bit different but does the same job. Since you specifically asked to learn about “PL/SQL Cursors,” we will teach you:

1. The **concept**, which is the same everywhere.
2. The **Oracle PL/SQL syntax** (SQL%FOUND, NO_DATA_FOUND, etc. — exactly as you listed them).
3. The **equivalent MySQL syntax**, which you can actually run on your installed MySQL/MariaDB server.

Changing the Statement Delimiter

MySQL normally uses ; to know when one statement ends. But inside a stored procedure, we need ; for lines *inside* the procedure too. So, before writing a procedure, we temporarily change MySQL’s “end of statement” symbol using DELIMITER:

```
DELIMITER //
```

```
CREATE PROCEDURE hello_world()  
BEGIN  
    SELECT 'Hello, Database World!' AS message;  
END //
```

```
DELIMITER ;
```

Now run it with:

```
CALL hello_world();
```

A Procedure With Parameters

```
DELIMITER //
```

```
CREATE PROCEDURE get_employees_by_dept(IN p_dept_id INT)  
BEGIN  
    SELECT first_name, last_name, salary  
    FROM employees  
    WHERE dept_id = p_dept_id;  
END //
```

```
DELIMITER ;
```

```
CALL get_employees_by_dept(1);
```

- IN p_dept_id INT means this parameter is an **input** — the caller gives a value in, and the procedure uses it.
- There is also OUT (procedure sends a value back out) and INOUT (both directions).

13.2 What Is a Cursor? (The Core Idea)

Normally, a SELECT statement returns a whole table of results all at once, to your application or screen. But sometimes, inside a stored procedure, we need to go through the results **one row at a time**, doing some logic on each row (like a loop, one step per row). A **cursor** is exactly this: a pointer that lets you walk through a result set, row by row.

There are two kinds: **implicit** cursors (automatic, made for you) and **explicit** cursors (you build them yourself, for multi-row processing).

13.3 Implicit Cursors (Oracle PL/SQL Concept)

In Oracle PL/SQL, whenever you run a SELECT INTO (a select that returns exactly one row) or a plain INSERT/UPDATE/DELETE, Oracle **automatically** creates a cursor behind the scenes for you — you never had to declare it yourself. This is called an **implicit cursor**.

Oracle gives you 4 special “attributes” you can check right after such a statement, to learn what happened:

Attribute	Meaning
SQL%FOUND	TRUE if the last statement affected/found at least one row
SQL%NOTFOUND	TRUE if the last statement affected/found zero rows
SQL%ROWCOUNT	The number of rows affected/found by the last statement
SQL%ISOPEN	Always FALSE for implicit cursors (Oracle opens and closes them instantly, automatically)

Oracle PL/SQL Example:

```
BEGIN
  UPDATE employees
  SET salary = salary * 1.05
  WHERE dept_id = 1;
```

```

IF SQL%FOUND THEN
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows were updated.');
```

```

ELSE
    DBMS_OUTPUT.PUT_LINE('No rows matched. Nothing was updated.');
```

```

END IF;
END;
/
```

This checks: did the UPDATE actually change anything? SQL%FOUND becomes TRUE if yes, and SQL%ROWCOUNT tells us exactly how many rows.

The MySQL Equivalent Idea

MySQL does not have SQL%FOUND / SQL%NOTFOUND exactly like Oracle. Instead, MySQL gives you the special variable ROW_COUNT(), which tells you how many rows were affected by the last statement:

```

DELIMITER //
```

```

CREATE PROCEDURE raise_dept_salary(IN p_dept_id INT)
BEGIN
    UPDATE employees
    SET salary = salary * 1.05
    WHERE dept_id = p_dept_id;

    IF ROW_COUNT() > 0 THEN
        SELECT CONCAT(ROW_COUNT(), ' rows were updated.') AS message;
    ELSE
        SELECT 'No rows matched. Nothing was updated.' AS message;
    END IF;
END //
```

```

DELIMITER ;

CALL raise_dept_salary(1);
```

This does the exact same job as the Oracle example: check if an UPDATE actually changed anything, and how many rows.

13.4 Explicit Cursors — Managing Multi-Row Processing Yourself

An **explicit cursor** is one **you** declare and control by hand, so you can loop through **many** rows, one at a time — something implicit cursors cannot do (implicit cursors are for single-row results only).

The Explicit Cursor Lifecycle (Oracle PL/SQL)

Every explicit cursor in Oracle PL/SQL follows these 4 steps:

1. **DECLARE** — define the cursor and its query.
2. **OPEN** — run the query, and get the cursor ready.
3. **FETCH** — grab one row at a time from the cursor, into variables.
4. **CLOSE** — release the cursor when done, freeing memory.

Oracle PL/SQL Example:

```
DECLARE
    CURSOR emp_cursor IS
        SELECT first_name, salary FROM employees WHERE dept_id = 1;

    v_first_name employees.first_name%TYPE;
    v_salary      employees.salary%TYPE;
BEGIN
    OPEN emp_cursor;

    LOOP
        FETCH emp_cursor INTO v_first_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;  -- stop the loop once no more rows

        DBMS_OUTPUT.PUT_LINE(v_first_name || ' earns ' || v_salary);
    END LOOP;

    CLOSE emp_cursor;
END;
```

Explanation, line by line:

- `CURSOR emp_cursor IS SELECT ...` — this **declares** the cursor and names its query, but does not run it yet.
- `v_first_name employees.first_name%TYPE;` — this declares a variable that automatically matches the data type of the `first_name` column. `%TYPE` is a handy Oracle trick that means “same type as this column.”
- `OPEN emp_cursor;` — this actually **runs** the query and prepares the results.
- `FETCH emp_cursor INTO v_first_name, v_salary;` — grabs the **next** row’s values into our variables.
- `EXIT WHEN emp_cursor%NOTFOUND;` — `%NOTFOUND` becomes `TRUE` once `FETCH` cannot find another row (we’ve reached the end). This line stops the loop at exactly the right moment.
- `CLOSE emp_cursor;` — releases the cursor’s resources. Always close what you open.

Cursor FOR Loops (Oracle PL/SQL — the Shortcut)

Oracle gives you a shortcut called a **cursor FOR loop**, which automatically opens, fetches (in a loop), and closes the cursor for you — you never write `OPEN`, `FETCH`, `EXIT WHEN`, or `CLOSE` by hand:

```

BEGIN
  FOR emp_row IN (SELECT first_name, salary FROM employees WHERE dept_id
    ↪ = 1) LOOP
    DBMS_OUTPUT.PUT_LINE(emp_row.first_name || ' earns ' ||
    ↪ emp_row.salary);
  END LOOP;
END;
/

```

This does the *exact same thing* as the longer example above, but in far fewer lines. emp_row becomes a “record” — you can access emp_row.first_name, emp_row.salary, matching the column names of your SELECT. Most Oracle developers use cursor FOR loops instead of manual OPEN/FETCH/CLOSE, whenever possible, because it is simpler and safer (you can never forget to CLOSE it).

The MySQL Equivalent: Cursors in Stored Procedures

MySQL also supports explicit cursors, inside stored procedures, with a very similar 4-step lifecycle: **DECLARE**, **OPEN**, **FETCH**, **CLOSE**. The big difference: MySQL does **not** have a %NOTFOUND attribute. Instead, MySQL uses something called a **HANDLER**, which “catches” the moment there are no more rows, similar to how try/catch works in other programming languages.

```

DELIMITER //

CREATE PROCEDURE list_dept_employees(IN p_dept_id INT)
BEGIN
  DECLARE v_first_name VARCHAR(50);
  DECLARE v_salary DECIMAL(10,2);
  DECLARE v_done INT DEFAULT FALSE;

  -- Step 1: DECLARE the cursor
  DECLARE emp_cursor CURSOR FOR
    SELECT first_name, salary FROM employees WHERE dept_id = p_dept_id;

  -- This HANDLER "catches" the end-of-data signal and sets v_done = TRUE
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

  -- Step 2: OPEN the cursor (runs the query)
  OPEN emp_cursor;

  read_loop: LOOP
    -- Step 3: FETCH one row into our variables
    FETCH emp_cursor INTO v_first_name, v_salary;

    IF v_done THEN
      LEAVE read_loop; -- stop the loop, similar to EXIT WHEN in
    ↪ Oracle
  END LOOP;
END;
//

```

```

        END IF;

        SELECT CONCAT(v_first_name, ' earns ', v_salary) AS info;
    END LOOP;

    -- Step 4: CLOSE the cursor
    CLOSE emp_cursor;
END //

DELIMITER ;

CALL list_dept_employees(1);

```

Notice the important MySQL rule: **all DECLARE statements (variables, cursor, and handler) must come at the top of the procedure**, in this order: variables first, then cursors, then handlers — before any other logic like OPEN or LOOP.

13.5 Parameterized Cursors — Passing Arguments Dynamically

A **parameterized cursor** accepts a parameter, so the same cursor definition can be reused with different filter values, instead of writing a new cursor for every department.

Oracle PL/SQL Example:

```

DECLARE
    CURSOR emp_cursor (p_dept_id NUMBER) IS
        SELECT first_name, salary FROM employees WHERE dept_id = p_dept_id;
BEGIN
    FOR emp_row IN emp_cursor(1) LOOP    -- pass department 1 as the argument
        DBMS_OUTPUT.PUT_LINE(emp_row.first_name);
    END LOOP;

    FOR emp_row IN emp_cursor(2) LOOP    -- reuse the SAME cursor,
        ↪ department 2 this time
        DBMS_OUTPUT.PUT_LINE(emp_row.first_name);
    END LOOP;
END;
/

```

MySQL Equivalent: MySQL cursors cannot take parameters directly inside their own DECLARE ... CURSOR FOR ... line. Instead, in MySQL, we simply use the **stored procedure's own parameter** inside the cursor's query (exactly like we already did in section 13.4, with p_dept_id). This achieves the same practical result — a reusable, parameter-driven cursor — just through the procedure's parameter, rather than a special cursor-parameter syntax.

13.6 Exception Handling With Cursors

Oracle PL/SQL: NO_DATA_FOUND and TOO_MANY_ROWS

These two are special **built-in exceptions** in Oracle PL/SQL, mainly triggered by SELECT INTO statements that expect exactly one row:

- **NO_DATA_FOUND**: raised automatically when a SELECT INTO finds **zero** rows (but expected one).
- **TOO_MANY_ROWS**: raised automatically when a SELECT INTO finds **more than one** row (but a SELECT INTO can only hold one row per variable).

```
DECLARE
    v_salary employees.salary%TYPE;
BEGIN
    SELECT salary INTO v_salary
    FROM employees
    WHERE emp_id = 999;    -- let's say this ID does not exist

    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found with that ID.');
```

↪

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one employee matched – expected
    only one.');
```

↪

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Some other unexpected error happened.');
```

```
END;
/
```

The EXCEPTION block works like a safety net — if any error happens in the BEGIN...END block above it, PL/SQL jumps down to the matching WHEN line, instead of crashing the whole program.

MySQL Equivalent: DECLARE ... HANDLER

MySQL uses **handlers** (which we already saw briefly in section 13.4) for all error handling — including the two Oracle-specific situations above.

```
DELIMITER //
```

```
CREATE PROCEDURE get_employee_salary(IN p_emp_id INT)
BEGIN
    DECLARE v_salary DECIMAL(10,2);
    DECLARE v_not_found INT DEFAULT FALSE;

    -- Similar purpose to NO_DATA_FOUND: catches "zero rows found"
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_not_found = TRUE;
```

```

-- A general safety-net handler, similar to Oracle's "WHEN OTHERS"
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    SELECT 'An unexpected database error occurred.' AS message;
END;

SELECT salary INTO v_salary
FROM employees
WHERE emp_id = p_emp_id
LIMIT 1;

IF v_not_found THEN
    SELECT 'No employee found with that ID.' AS message;
ELSE
    SELECT CONCAT('Salary: ', v_salary) AS message;
END IF;
END //

DELIMITER ;

CALL get_employee_salary(999);

```

Two kinds of MySQL handlers:

- **CONTINUE HANDLER:** catches the problem, then **keeps running** the rest of the procedure, right after the statement that caused it.
- **EXIT HANDLER:** catches the problem, then **immediately stops** the current BEGIN...END block (similar to how Oracle's EXCEPTION block stops normal execution and jumps to a WHEN clause).

Note on "TOO_MANY_ROWS" in MySQL: MySQL's plain SELECT ... INTO does not raise a special "too many rows" error the way Oracle does — if more than one row matches, MySQL puts the **last** matching row's value into the variable, silently. This is why, in real MySQL code, we protect ourselves by adding LIMIT 1 (as shown above) or by using an explicit cursor/loop instead, whenever more than one row might match.

13.7 Quick Summary: Cursor Concepts, Oracle vs MySQL

Concept	Oracle PL/SQL	MySQL
Implicit cursor status	SQL%FOUND, SQL%NOTFOUND, SQL%ROWCOUNT	ROW_COUNT()
Declare explicit cursor	CURSOR c IS SELECT ...	DECLARE c CURSOR FOR SELECT ...
Open cursor	OPEN c;	OPEN c;

Concept	Oracle PL/SQL	MySQL
Fetch a row	FETCH c INTO var1, var2;	FETCH c INTO var1, var2;
Detect end of rows	c%NOTFOUND	DECLARE ... HANDLER FOR NOT FOUND
Close cursor	CLOSE c;	CLOSE c;
Shortcut loop	Cursor FOR loop	No direct equivalent; use LOOP + handler
Handle “no row found” error	EXCEPTION WHEN NO_DATA_FOUND	DECLARE HANDLER FOR NOT FOUND
Handle “too many rows” error	EXCEPTION WHEN TOO_MANY_ROWS	Not automatic; use LIMIT 1 or a cursor/loop
General catch-all error	EXCEPTION WHEN OTHERS	DECLARE EXIT HANDLER FOR SQLEXCEPTION

Chapter 14: Database Triggers

14.1 What Is a Trigger?

A **trigger** is a piece of code that the database runs **automatically**, whenever a certain event happens to a table — like an INSERT, UPDATE, or DELETE. You never “call” a trigger yourself; the database calls it for you, at the right moment, automatically.

Think of it like a doorbell: you don’t press it yourself from inside the house — a *visitor* pressing the button is the “event” that automatically rings the bell (the “trigger”).

14.2 Trigger Structure — The Basic Pieces

Every trigger needs 4 pieces of information:

1. **Timing:** BEFORE or AFTER the event.
2. **Event:** INSERT, UPDATE, or DELETE.
3. **Table:** which table it watches.
4. **Level:** row-level (FOR EACH ROW) or statement-level.

14.3 BEFORE vs AFTER vs INSTEAD OF

- **BEFORE trigger:** runs *before* the actual change happens. Common use: check or clean up data, or block a bad change, before it’s saved.
- **AFTER trigger:** runs *after* the actual change has already been saved. Common use: logging/auditing, updating a related table, sending a notification.
- **INSTEAD OF trigger:** runs *instead of* the actual event — mainly used on **views** (especially views built from complex joins), to define custom logic for what an INSERT/UPDATE/DELETE on that view should actually do.

Important MySQL note: MySQL supports BEFORE and AFTER triggers, but **does not support INSTEAD OF triggers** (some other databases, like Oracle and SQL Server, do support them, mainly for views). We explain the concept here (because you asked for it), but you cannot create an actual INSTEAD OF trigger in MySQL.

14.4 Row-Level vs Statement-Level Triggers

- **Row-level trigger** (FOR EACH ROW): runs **once for every single row** affected by the statement. If an UPDATE changes 100 rows, a row-level trigger runs 100 times — once per row.
- **Statement-level trigger:** runs **only once**, no matter how many rows were affected by the statement (even if 0 rows or 1,000 rows were affected).

Important MySQL note: MySQL only supports **row-level** triggers (FOR EACH ROW is actually required in the syntax). Some other databases (like Oracle) let you choose statement-level triggers too. We’ll show both concepts, but MySQL code examples will always be row-level.

14.5 Correlation Identifiers: NEW and OLD

Inside a row-level trigger, you often need to look at the actual data of the row being changed — either its value **before** the change, or its value **after** the change. Special “pseudo-records” let you do this:

- **OLD**: the row’s values **before** the change (only available for UPDATE and DELETE — there is no “before” for an INSERT, since the row didn’t exist yet).
- **NEW**: the row’s values **after** the change (only available for INSERT and UPDATE — there is no “after” for a DELETE, since the row won’t exist anymore).

Oracle PL/SQL syntax uses a colon in front: :NEW.column_name and :OLD.column_name.

MySQL syntax does **not** use a colon: just NEW.column_name and OLD.column_name.

Event	OLD available?	NEW available?
INSERT	No	Yes
UPDATE	Yes	Yes
DELETE	Yes	No

14.6 Writing DML Triggers in MySQL

Example 1: BEFORE INSERT Trigger — Validating/Cleaning Data

Let’s say we want to make sure no employee is ever inserted with a negative salary — instead of rejecting it completely, let’s force it to 0 automatically.

```
DELIMITER //
```

```
CREATE TRIGGER trg_check_salary_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SET NEW.salary = 0;
    END IF;
END //
```

```
DELIMITER ;
```

Because this is a BEFORE trigger, we are allowed to **change** NEW.salary — the corrected value is what actually gets saved. (In an AFTER trigger, you can *read* NEW, but you cannot change it — the row is already saved by then.)

Example 2: BEFORE UPDATE Trigger — Preventing an Invalid Change

Let’s block anyone from *lowering* an employee’s salary through a normal UPDATE (maybe salary decreases must go through a special approval process elsewhere).

```

DELIMITER //

CREATE TRIGGER trg_prevent_salary_decrease
BEFORE UPDATE ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Salary cannot be decreased through a normal
            ↪ update.';
    END IF;
END //

DELIMITER ;

```

SIGNAL SQLSTATE '45000' is how MySQL raises a **custom error**, which cancels the whole UPDATE statement, and shows your custom message text to whoever tried to run it. This is a very common way to “prevent invalid mutations” (stop a bad change before it happens), exactly as mentioned in your topic list.

Example 3: AFTER INSERT Trigger — Data Auditing / History Tracking

This is one of the most common real-world uses of triggers: automatically keeping a **history/audit log**, without the application even knowing it’s happening.

First, we make a log table:

```

CREATE TABLE salary_audit_log (
    log_id      INT PRIMARY KEY AUTO_INCREMENT,
    emp_id      INT,
    old_salary  DECIMAL(10,2),
    new_salary  DECIMAL(10,2),
    changed_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

Now, an AFTER UPDATE trigger that logs every salary change:

```

DELIMITER //

CREATE TRIGGER trg_log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary != NEW.salary THEN
        INSERT INTO salary_audit_log (emp_id, old_salary, new_salary)
        VALUES (OLD.emp_id, OLD.salary, NEW.salary);
    END IF;
END //

```

```
DELIMITER ;
```

Now, test it:

```
UPDATE employees SET salary = 60000 WHERE emp_id = 1;
```

```
SELECT * FROM salary_audit_log;
```

You will see a row automatically appear in `salary_audit_log`, even though we never wrote any `INSERT` statement ourselves — MySQL's trigger did it for us, silently, in the background. This is exactly what “automating business logic” means.

Example 4: AFTER DELETE Trigger — Keeping a Record of Deleted Data

```
CREATE TABLE deleted_employees_log (  
    log_id          INT PRIMARY KEY AUTO_INCREMENT,  
    emp_id          INT,  
    first_name      VARCHAR(50),  
    last_name       VARCHAR(50),  
    deleted_at      DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_log_employee_delete  
AFTER DELETE ON employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO deleted_employees_log (emp_id, first_name, last_name)  
    VALUES (OLD.emp_id, OLD.first_name, OLD.last_name);  
END //
```

```
DELIMITER ;
```

Notice: for a `DELETE` trigger, only `OLD` exists (the row that used to be there) — there is no `NEW`, because after a delete, there is nothing “new.”

14.7 Practical Applications Recap

- **Data auditing / history tracking:** Examples 3 and 4 above — automatically saving a record of every change, without changing your application code at all.
- **Complex integrity checks:** checks that go beyond a simple `CHECK` constraint, like comparing a new value against an old value, or checking values across more than one column/table.
- **Preventing invalid mutations:** Example 2 above — blocking a change before it happens, with a custom error message, using `SIGNAL SQLSTATE`.

14.8 Managing Triggers

```
-- See all triggers in the current database  
SHOW TRIGGERS;
```

```
-- Remove a trigger  
DROP TRIGGER trg_log_salary_change;
```

Important warning: Triggers run silently, in the background. If you have many triggers, it can become hard to understand *why* something changed, just by reading the application code — you also have to remember to check the triggers. Use triggers for genuinely automatic things (like audit logs), but don't hide your main business logic entirely inside triggers, or your system becomes hard to debug.

Chapter 15: Stored Functions vs Stored Procedures

15.1 The Difference

- A **stored procedure** is called with CALL, and does not have to return a value directly (though it can return result sets or use OUT parameters).
- A **stored function** must return exactly **one value**, and can be used directly inside a SELECT statement, just like a built-in function such as AVG().

```
DELIMITER //
```

```
CREATE FUNCTION get_yearly_salary(p_emp_id INT)
RETURNS DECIMAL(12,2)
DETERMINISTIC
BEGIN
    DECLARE v_salary DECIMAL(10,2);

    SELECT salary INTO v_salary FROM employees WHERE emp_id = p_emp_id
    ↪ LIMIT 1;

    RETURN v_salary * 12;
END //
```

```
DELIMITER ;
```

Now use it directly inside a query:

```
SELECT first_name, get_yearly_salary(emp_id) AS yearly_salary
FROM employees;
```

DETERMINISTIC tells MySQL “this function always returns the same output for the same input” — this helps MySQL optimize how it runs the function.

Chapter 16: Transactions and Concurrency (ACID)

16.1 What Is a Transaction?

A **transaction** is a group of one or more SQL statements that must **all succeed together, or all fail together** — there is no “half-done” state allowed. This matters a lot for things like bank transfers: moving money **out** of one account and **into** another must either both happen, or neither happen.

16.2 COMMIT and ROLLBACK

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;  -- take
    ⇨ money out
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;  -- put
    ⇨ money in

COMMIT;  -- save both changes permanently
```

If something goes wrong partway through (like the second UPDATE failing), you can **undo everything** done since START TRANSACTION, using:

```
ROLLBACK;
```

Example with a safety check:

```
START TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;

-- Check if this caused a negative balance (not allowed!)
SELECT balance FROM accounts WHERE account_id = 1;
-- (imagine we check this in our application code, and see balance is now
    ⇨ negative)

ROLLBACK;  -- undo the update above, since it broke our rule
```

16.3 ACID — The 4 Properties of a Good Transaction

Letter	Word	Simple Meaning
A	Atomicity	All statements in a transaction happen together, or none happen at all. No “halfway.”
C	Consistency	A transaction can only move the database from one valid state to another valid state (it can’t break your rules/constraints).

Letter	Word	Simple Meaning
I	Isolation	Transactions running at the same time should not mess with each other's in-progress (uncommitted) data.
D	Durability	Once a transaction is committed, it is saved permanently — even if the power goes out right after.

16.4 Isolation Levels (Just the Basics)

Isolation levels control exactly how much one unfinished transaction can “see” of another unfinished transaction. From least strict to most strict:

1. **READ UNCOMMITTED** — can see other transactions' uncommitted (unsaved) changes. Risky (“dirty reads”).
2. **READ COMMITTED** — only sees other transactions' changes after they are committed.
3. **REPEATABLE READ** — (MySQL's default) guarantees that if you read the same row twice in one transaction, you get the same result both times.
4. **SERIALIZABLE** — the strictest; transactions behave as if they ran one at a time, in a strict order.

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Higher strictness = safer, but usually slower (because the database has to do more work to protect transactions from each other).

Chapter 17: Basic Database Security

17.1 Users and Privileges

```
-- Create a new user with a password
CREATE USER 'report_viewer'@'localhost' IDENTIFIED BY 'a_strong_password';

-- Give that user permission to only READ (SELECT) from company_db
GRANT SELECT ON company_db.* TO 'report_viewer'@'localhost';

-- Give full permissions on one specific table
GRANT SELECT, INSERT, UPDATE, DELETE ON company_db.employees TO
  ↪ 'report_viewer'@'localhost';

-- Apply the changes immediately
FLUSH PRIVILEGES;
```

17.2 Removing Privileges

```
REVOKE INSERT, UPDATE, DELETE ON company_db.employees FROM
  ↪ 'report_viewer'@'localhost';
```

17.3 Deleting a User

```
DROP USER 'report_viewer'@'localhost';
```

17.4 Good Security Habits

- Never give the root account to your applications — create a separate, limited user for each application.
- Only grant the exact privileges needed (this is called the **Principle of Least Privilege**).
- Use views (Chapter 10) to hide sensitive columns from certain users.
- Always use parameterized queries in your application code (not shown here, since that is more of a programming-language topic) to prevent **SQL Injection** attacks — never build SQL strings by directly gluing user input into them.

Chapter 18: Performance Basics

18.1 Using EXPLAIN

EXPLAIN shows you **how** MySQL plans to run a query, without actually running it — very useful for finding slow spots.

```
EXPLAIN SELECT * FROM employees WHERE last_name = 'Hossain';
```

Look at the type and rows columns in the output:

- If type says ALL, MySQL is scanning the **entire table**, row by row (slow on big tables) — this often means you need an index.
- The rows column estimates how many rows MySQL expects to check.

18.2 General Tips

- Add indexes on columns you frequently search (WHERE) or join (JOIN ... ON) on.
- Avoid SELECT * in real applications — only select the columns you actually need.
- Avoid putting a function around an indexed column in WHERE (like WHERE YEAR(hire_date) = 2022) — this can stop MySQL from using the index. Instead, write it as a range: WHERE hire_date BETWEEN '2022-01-01' AND '2022-12-31'.
- Use LIMIT when you only need a portion of the results.
- Regularly check slow queries using MySQL's slow query log, for real production systems.

Chapter 19: Practice Project — Build a Mini Store Database

Use this chapter to test everything you learned. Try to write each query **yourself first**, before looking at the answer.

19.1 Setting Up the Project

```
CREATE DATABASE mini_store;
USE mini_store;

CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name        VARCHAR(100) NOT NULL,
    city        VARCHAR(50)
);

CREATE TABLE products (
    product_id   INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    price        DECIMAL(10,2) CHECK (price > 0),
    category     VARCHAR(50)
);

CREATE TABLE orders (
    order_id     INT PRIMARY KEY AUTO_INCREMENT,
    customer_id  INT,
    order_date   DATE,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_id     INT,
    product_id   INT,
    quantity     INT CHECK (quantity > 0),
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

INSERT INTO customers (name, city) VALUES
('Rahim', 'Dhaka'), ('Karim', 'Sylhet'), ('Nadia', 'Dhaka');

INSERT INTO products (product_name, price, category) VALUES
('Notebook', 40.00, 'Stationery'),
('Pen', 10.00, 'Stationery'),
('Laptop Bag', 900.00, 'Accessories'),
('USB Cable', 150.00, 'Electronics');
```

```

INSERT INTO orders (customer_id, order_date) VALUES
(1, '2026-01-05'), (2, '2026-01-10'), (1, '2026-02-01'), (3, '2026-02-15');

INSERT INTO order_items (order_id, product_id, quantity) VALUES
(1, 1, 5), (1, 2, 10),
(2, 3, 1),
(3, 4, 2), (3, 2, 3),
(4, 1, 1), (4, 4, 1);

```

19.2 Practice Tasks (Try Them Yourself!)

1. List every customer's name and city. *(Chapter 4)*
2. Find all products cheaper than 100 taka. *(Chapter 4)*
3. Show every order along with the customer's name. *(Chapter 7, JOIN)*
4. Show the total quantity sold, per product. *(Chapter 8, GROUP BY)*
5. Find customers who have placed more than 1 order. *(Chapter 8, HAVING)*
6. Find products that were never ordered. *(Chapter 9, subquery with NOT IN, or Chapter 7 LEFT JOIN with IS NULL)*
7. Rank customers by how much money they've spent in total (Hint: you'll need to join order_items and products to calculate money spent, then use RANK()). *(Chapter 12)*
8. Write a stored procedure get_customer_orders(customer_id) that lists all orders for one customer. *(Chapter 13)*
9. Write a trigger that automatically prevents an order_items row from having a quantity greater than 1000. *(Chapter 14)*
10. Write a query showing a running total of orders placed, by date, using a window function. *(Chapter 12)*

19.3 Sample Answers (Check Your Work)

```

-- 3
SELECT o.order_id, c.name, o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;

-- 4
SELECT p.product_name, SUM(oi.quantity) AS total_sold
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_name;

-- 5
SELECT customer_id, COUNT(*) AS num_orders
FROM orders
GROUP BY customer_id

```

```

HAVING COUNT(*) > 1;

-- 6
SELECT p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;

-- 7
SELECT c.name,
       SUM(oi.quantity * p.price) AS total_spent,
       RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS
       ↪ spending_rank
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY c.customer_id, c.name;

-- 10
SELECT order_id, order_date,
       COUNT(*) OVER (ORDER BY order_date) AS running_order_count
FROM orders
ORDER BY order_date;

```

Try tasks 1, 2, 8, and 9 fully on your own — they use ideas already fully explained in earlier chapters.

Appendix A: Quick Reference Cheat Sheet

Basic Queries

```
SELECT col1, col2 FROM table_name WHERE condition ORDER BY col1 LIMIT n;
INSERT INTO table_name (col1, col2) VALUES (val1, val2);
UPDATE table_name SET col1 = val1 WHERE condition;
DELETE FROM table_name WHERE condition;
```

Table Management

```
CREATE TABLE t (id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50) NOT
↪ NULL);
ALTER TABLE t ADD COLUMN new_col INT;
ALTER TABLE t DROP COLUMN new_col;
DROP TABLE t;
```

Joins

```
SELECT * FROM a JOIN b ON a.id = b.a_id;           -- INNER JOIN
SELECT * FROM a LEFT JOIN b ON a.id = b.a_id;      -- keep all of a
SELECT * FROM a RIGHT JOIN b ON a.id = b.a_id;     -- keep all of b
```

Grouping

```
SELECT col, COUNT(*), AVG(x) FROM t GROUP BY col HAVING COUNT(*) > 1;
```

Window Functions

```
ROW_NUMBER() OVER (PARTITION BY col ORDER BY col2)
RANK()          OVER (PARTITION BY col ORDER BY col2)
DENSE_RANK()   OVER (PARTITION BY col ORDER BY col2)
LAG(col, n)     OVER (ORDER BY col2)
LEAD(col, n)    OVER (ORDER BY col2)
SUM(col) OVER (PARTITION BY p ORDER BY o ROWS BETWEEN 2 PRECEDING AND
↪ CURRENT ROW)
```

Stored Procedure Skeleton

```
DELIMITER //
CREATE PROCEDURE proc_name(IN p1 INT, OUT p2 INT)
BEGIN
```

```

    -- body
END //
DELIMITER ;
CALL proc_name(1, @result);

```

Cursor Skeleton (MySQL)

```

DECLARE v_done INT DEFAULT FALSE;
DECLARE cur CURSOR FOR SELECT col FROM t;
DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;
OPEN cur;
loop_label: LOOP
    FETCH cur INTO v_var;
    IF v_done THEN LEAVE loop_label; END IF;
    -- process v_var
END LOOP;
CLOSE cur;

```

Trigger Skeleton (MySQL)

```

CREATE TRIGGER trg_name
BEFORE INSERT ON table_name
FOR EACH ROW
BEGIN
    -- use NEW.col / OLD.col here
END;

```

Transactions

```

START TRANSACTION;
-- statements
COMMIT;    -- or ROLLBACK;

```

Users and Privileges

```

CREATE USER 'u'@'localhost' IDENTIFIED BY 'pw';
GRANT SELECT, INSERT ON db.* TO 'u'@'localhost';
REVOKE INSERT ON db.* FROM 'u'@'localhost';

```

Appendix B: Where to Go Next

You now know SQL and Database Systems from the very basics all the way to cursors, triggers, and window functions. To keep growing:

1. **Practice daily** on the `company_db` and `mini_store` databases we built. Try inventing your own new questions and answering them with SQL.
2. **Build a small real project** — for example, a simple library system, a hostel management system, or an online shop — design the tables yourself, apply normalization, then add triggers and stored procedures.
3. **Learn a programming language's database connector** (like Python's `mysql-connector-python`, or PHP's `PDO`) to connect a real application to your MySQL database.
4. **Explore MySQL Workbench's tools** — the visual EER (Entity-Relationship) diagram designer is a great way to design databases visually before writing SQL.
5. **Read MySQL's official documentation** whenever you're unsure of exact syntax — it is free and very detailed.

Good luck, Arif. Keep practicing, and you will get comfortable with all of this very quickly.